

Overview of current text-to-speech techniques: Part I — text and linguistic analysis

M Edgington, A Lowry, P Jackson, A P Breen and S Minnis

Speech synthesis is an increasingly important technique in interactive voice systems, as shown elsewhere in this issue, and is crucial to the next generation of spoken language dialogue systems. Described here are many of the techniques and algorithms used in state-of-the art commercial text-to-speech synthesisers, where real world issues of robustness, reliability and processing cost compete with the specific requirements of naturalness and intelligibility. This paper describes techniques used in text and linguistic analysis, which build up a linguistic structure defining the important features of the input text. This linguistic structure is then used by the prosody and speech generation processes to create the synthetic speech signal.

1. Introduction

Speech is the primary communication method of human beings, evolving in *Homo Sapiens* at least 100 000 years ago, making it the most natural powerful expressive medium between people. In contrast, writing has been part of human culture for only a few thousand years, and originated as a way of permanently recording information, since speech is an inherently transient phenomenon. In the modern world, extremely large amounts of information are stored electronically. A method converting this electronically stored text, in any major language, into high-quality natural-sounding speech would provide an extremely useful tool for many applications. This is the long term goal of many speech scientists and engineers, although current text-to-speech (TTS) systems have a long way to go to reach this goal. However, commercial systems, such as BT's Laureate [1], are capable of producing highly intelligible speech which, under ideal conditions, has a naturalness approaching that of a human speaker [2].

The last decade has seen a substantial improvement in the naturalness of synthetic speech, due largely to the availability of high-quality speech concatenation algorithms [3]. The resulting improvement in segmental quality has provided an impetus to the development of more complex and subtle intonation and phrasing models. This paper presents an overview of the algorithmic techniques used by state-of-the-art commercial TTS systems in converting from

the input text stream to the final speech signal. Another paper [1] describes major applications of TTS synthesis and details a particular system under development at BT called Laureate. The authors have all been actively involved in the development of the BT Laureate system, and this paper and the next consequently have a slight bias towards the methods used in that system, though they focus on techniques that can be universally applied. A description of earlier work in TTS synthesis can be found in the excellent review paper by Klatt [4] which covers the fundamentals, and developments up to 1987.

The focus of this paper (and its companion [5]) is on 'real world' commercial TTS systems, rather than systems designed purely for research. In this arena some issues rarely addressed in research-only systems become significant. In particular, robustness, maintainability and computational complexity of algorithms are important to commercial developers, while the primary motivation for a research-only system is often to provide a test bed for a particular underlying theory. This is not to say that a commercial system will not employ the latest research -- for example, BT's Laureate TTS was designed explicitly to accommodate different theoretical models within its processing architecture to encourage its use within the research community.

Multilingual TTS systems are becoming increasingly important, especially within Europe, and although this paper concentrates on English, most of the techniques described are equally applicable to many other languages.

1.1 Overview of TTS conversion process

The process of generating synthetic speech from unrestricted text is essentially a two-stage process, as shown in Fig 1.

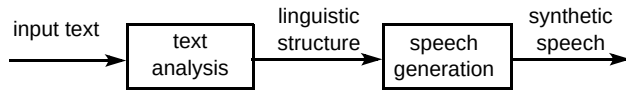


Fig 1 Text-to-speech as an analysis and synthesis process.

- The first stage analyses the input text to determine its underlying linguistic structure, including the sound sequence, or phonemic representation, of every word in the text. This stage is sometimes referred to as ‘text-to-phoneme’ conversion, but this does not adequately describe the whole process, as it neglects other parts of the linguistic structural analysis.
- The second stage synthesises a speech waveform from this linguistic structure. This synthesis step can be further subdivided into two steps, that of prosody generation which generates the intonation and rhythm of the speech, and speech generation, which produces the final speech waveform. This whole stage is sometimes referred to as ‘phoneme-to-speech’ conversion in the literature.

Consequently, the subject is split into two papers, following the structure of Fig 1. Part I, which includes this introduction, describes techniques for text and linguistic analysis, and Part II [5] describes techniques for the automatic generation of prosody and the speech signal.

2. Text analysis overview

The text analysis stage of a TTS system takes the input text and analyses it to generate a linguistic description and pronunciation for the whole text. This information is used by the prosody generation and speech synthesis stages to produce the final speech. Traditionally, much research effort has gone into the speech generation aspects of the TTS process, while text analysis for TTS systems has received less attention. However, as the quality of synthesis for phrases and sentences has improved, it has become increasingly clear that the text analysis stage is crucial in improving and maintaining TTS performance for longer stretches of text covering many sentences or paragraphs [2].

The text analysis stage can be split into the following four tasks.

Text segmentation and normalisation — the input text must be split into a hierarchy of segments: at the highest level are paragraphs, which are split into sentences, which are further divided into words. Decisions have to be made at each stage to determine the exact boundaries. In parallel with this segmentation, the text is normalised which involves the conversion of abbreviations, digit sequences, and other symbols into a predefined format, and introduces a standard treatment of punctuation. It is essential that the text segmentation and normalisation stage works correctly for unrestricted text, as the rest of the TTS relies on the output of this stage.

Morphology and pronunciation — pronunciations must be generated for every word in the text so that they can be spoken out. This is partly achieved by using a pronouncing dictionary or lexicon, but unfortunately most languages have a very large and ever-increasing number of words, making it impossible to produce an absolutely complete dictionary. Therefore the dictionary is supplemented by one or more rule- or statistical-based pronunciation methods. To aid this process, a morphological (or structural) analysis of the words is usually performed. This determines the ‘root form’ of every word, e.g. love from loves, and thus it allows the dictionary to store just headword entries, rather than all derived forms of a word. At this stage other word-level information is normally generated, such as part-of-speech (or syntactic) and possibly lexical semantic information. Certain sequences of words may also be merged into short lexical phrases, for example it is easier to treat the phrase ‘to and fro’ as a single unit, than as a sequence of three separate words.

Tagging and parsing — the morphology and pronunciation tasks produce a sequence of words (or lexical phrases), each with a pronunciation and some syntactic information. For further analysis it is necessary to consider the overall structure of the sentence, which can give a clue to its meaning. The first part of this task is to unambiguously determine the part-of-speech for each word in the sentence, a process known as tagging. This has the added benefit that most homographs (words spelt the same way but with different pronunciations) differ in their part-of-speech. For example, ‘live’ rhyming with ‘give’ is used as a verb, while ‘live’ rhyming with ‘hive’ is used as an adjective. Thus, in most cases, if the part-of-speech for the word is known, the correct pronunciation should also be known. The second part of the task is to build up a syntactic parse which represents the sentence structure. The syntactic parse represents the structural meaning of the sentence, so it can identify phrases, which phrases are linked, and how they are linked within a hierarchy.

Continuous speech effects — the pronunciation task produces phoneme transcriptions for individual words (or lexical phrases), as if each word were spoken in isolation. However, there are many processes in continuous speech

which operate across word boundaries, or modify the isolated word pronunciation in some other way, dependent on the syntactic function of the word in the sentence. The so-called weak forms (consider the difference in pronunciation of 'the' in 'the apple' and 'the car') are an example of the latter class of processes. It is important to model these effects in high-quality TTS systems, otherwise the synthetic speech can sound over-articulated and stilted.

2.1 *The variability of input text*

In a truly unrestricted TTS system, there are very few strong assumptions that can be made about the input text, other than the basic character encoding (e.g. ASCII). A text stream is essentially flat and linear, but is normally used to convey information that has a structure, so a number of 'orthographic conventions' are employed by the writer. Some of these conventions are well-defined, such as marking the end of a sentence with a full stop; others are rather more ill-defined, such as the use of digits rather than words, and the use of intra-sentence punctuation [7]. For example, sentences a to d below have different punctuation and number formats, yet have exactly the same meaning. This doesn't mean that punctuation can be ignored, since sentence e only differs from a in commas, yet has a significantly different meaning. (The first four sentences give the husband's age as additional information, while the last implies polygamy; it is the twenty seven year old husband who has left, not one of the others -- the first four sentences have a 'non-defining relative clause', but the last has a 'defining relative clause'.) This distinction can be clearly made when speaking by using a different intonation for each sentence type, and inserting a pause before the relative pronoun, 'who'.

- a My husband, who is 27, has left me.
- b My husband, who is twenty-seven, has left me.
- c My husband -- who is twenty seven -- has left me.
- d My husband (who is 27) has left me.
- e My husband who is 27 has left me.

The traditions of the original media, and the formality of the text can also have an effect on which conventions are used. Figure 2 shows three text extracts: from a local newspaper, a personal e-mail, and a novel — the original formatting has been retained as far as possible. Most readers should easily identify which extract comes from which source, from their content, style and format. It is clear that different conventions are used in each extract.

The newspaper text (Fig 2 (a)) uses different fonts to distinguish the headline from the article, and typically there is no punctuation in the headline. The first word of the article is written entirely in capitals, new paragraphs are indicated by first line indentation, and there are often many

words broken across lines, such as the word 'conditions' in the first paragraph. It should be noted that large local and national newspapers are produced using an editorial style guide, so there is consistency between all articles in a particular paper, but not necessarily between papers.

There is an interesting contrast in the e-mail (Fig 2 (b)) between the strict format of the header and the very informal, poorly punctuated text. Two typical problems in e-mail occur in the extract, the infamous 'smiley' face (used as an irony marker) and the signature footer, which often contains ASCII graphics. The specialised format of e-mail lends itself to the use of a preprocessor to extract the important header information and the message body [8].

The extract from the novel (Fig 2 (c)) is very similar to what would be regarded as 'normal' text, although this extract was chosen to show some of the features introduced by written dialogue. The dialogue seen in this extract is mainly written in complete sentences (unlike real speech) and has a different style to the quotations that appear in journalism.

The role of the text normalisation process is thus to impose some kind of order upon the unpredictable input. This then allows the rest of the system to be insulated from the 'messy' world of the original text.

There are two basic, and opposing, approaches to the normalisation problem.

In the first approach, a fairly simple normalised text format is defined, to which the input text must conform, and the normalisation problem is pushed back on to the application calling the TTS. For example, the normalised form may allow only lower-case letters within words, spaces between words, very limited punctuation, and only full stops or question marks at the end of the sentence. At first sight this appears to be merely avoiding the problem, but the justification of this approach is that the application generating the text has the most comprehensive knowledge about the format of the text with which it is dealing. For example, the abbreviations used in pharmaceutical research are different from financial market information or directory enquiries.

The second approach is to allow unrestricted text into the system, and employ a set of rules to make decisions about the intended 'meaning' of the text. These rules may be switched on or off by the application. The major disadvantage here, is that the TTS cannot predict all possible text formatting issues, and may make wrong decisions.

In essence the two approaches outlined above define the role of a TTS system differently -- how much implicit application knowledge must the TTS have to perform its job? In practical systems a compromise between the two extremes is usually taken, with a fairly intelligent normalisation front end controlled by options set by the

Pot-plant tree has peach of a summer

WHILE many plants are withering in the dry conditions this summer a couple from Snape have finally seen their peach tree emerge in all its glory.

John and Jane Ling planted a peach stone in a pot about eight years ago.

It was later planted in the garden but, as far as fruits were concerned, it was something of a disappointment

(a)

From mde@paris Mon Jun 26 09:59 BST 1995
Received: by paris
(1.3.99.11/1.22) id AA0194; Mon, 26 Jun 1995 09:59:54 +0100
From: Mike Edgington <mde@paris>
Return-Path: <mde@paris>
Subject: Gloat
To: pj@paris, devaney@paris, pd, guillem@paris
Date: Mon, 26 June 95 9:59:54 BST
Mailer: Elm [revision: 70.85]
Status: RO

Dear all,

Oh dear, I seem to have won our little rugby world-cup competition.
you can all congratulate me when I get back :-)

mike.

=====

Mike Edgington, Speech Technology	Tel: +44 1473 640000
BT Labs, Ipswich, UK.	E-mail: mde@dwarf.bt.co.uk

(b)

He held out his hand, and I saw in the light of the lamp that two of his knuckles were burst and bleeding.

"It's not an airy nothing, you see," said he, smiling. "On the contrary, it is solid enough for a man to break his hand over. Is Mrs Watson in?"

"She is away upon a visit."

"Indeed! You are alone?"

"Quite."

"Then it makes it the easier for me to propose that you should come away with me for a week to the Continent."

"Where?"

"Oh, anywhere. It's all the same to me."

There was something very strange in all this. It was not Holmes's nature to take an aimless holiday, and something about his pale, worn face told me that his nerves were at their highest tension. He saw the question in my eyes, and, putting his finger-tips together and his elbows upon his knees, he explained the situation.

"You have probably never heard of Professor Moriarty?" said he.

(c)

Fig 2 Examples of three text extracts.

application, for example from an expanded abbreviation list. For most text there would be no need for an application to filter it, unless the format were very particular or unusual, e.g. e-mail. In this way the onus of text formatting is shared between the application and the TTS system.

2.2 Text segmentation

The most basic function of the text analysis process is to segment the input text into manageable units, and these units must also have some place within the linguistic framework(s) used in the rest of the TTS system. It is usual to split the text into sentences, and each sentence into individual words. Higher level structures such as paragraphs and possibly even larger units may also need to be identified. Below the sentence level, it is necessary to split the text into words, and possibly 'word groups'. (Word group is used to mean a group of words that are created to replace a non-letter sequence in the original text, e.g. the words 'seven hundred and four' for the input text '704'.) This issue of intra-sentence segmentation is closely tied to text normalisation, as described below.

For English at least, the most invariant substantial unit of written text is the sentence. Sentences are easy to segment most of the time, since they normally start with a capitalised word and end with a full stop or other termination character (question mark, exclamation mark or ellipsis). This simple approach produces reasonable results, but does not account for more complex cases where the presence of a termination character does not signal a sentence boundary. In the examples below, a full stop may be part of a number, a, date, b, acronym, c, or indicate an abbreviation, d. Most of these cases (a,b) can be resolved by considering the context immediately following the full stop; if it is a space, or certain other punctuation, e, then the sentence is terminated.

- a Please pay \$43.23 every month.
- b Arrange the meeting for 23.08.95 in room 6.
- c Kilby was an M.I.5 informant.
- d How about Mon. 23rd?
- e He said, "I love you."

Acronyms, *f*, and especially abbreviations can still cause considerable problems, since they may also be potential words, as in the rather extreme example, *g*.

f Colin did leave the N.A.S.A. building on time!

g He tried to walk on the Sun. Howard died.

Without understanding the context it is impossible for even a human to tell whether the male subject of *g* was recovering from an accident on the Sunday that Howard passed away, or if Howard's foolhardy attempt at solar perambulation proved fatal! This example is particularly tricky due to the secondary cue of an initial capital letter on the word following the potential boundary. Decisions about sentence boundaries in such cases will be arbitrary unless the system has some knowledge of the text's meaning.

The final case to consider is where a sentence boundary occurs but there is no terminating character, as is often the case in electronic media such as e-mail, and some informal texts. In general the presence of a blank line or end of file implicitly marks a sentence boundary, as would systematic change in visual format, e.g. the change of font in the newspaper extract in Fig 2(a).

Paragraphs and sections

For passages longer than four or five sentences, text is often split into larger logical groupings, typically paragraphs and sections, and this logical structure should be made available to the rest of the TTS system. Information about paragraph structure is necessary to model long domain intonation [9] and inter-paragraph pausing, which is longer than inter-sentence pausing. Paragraph boundary information is also useful in trying to resolve pronoun cross-references, which occur infrequently across paragraph boundaries.

Paragraph breaks are generally indicated by a blank line, or by some form of initial indentation, immediately following a sentence boundary. There are circumstances where these features do not indicate a paragraph break, such as some list formats.

2.3 Text normalisation

Text normalisation is sometimes referred to as text formatting, but this somewhat undervalues the nature of the task. It is in essence a process of 'de-formatting' the text, or unwrapping the strict token sequence from the visual presentation style, and encoding the useful parts of the 'style' in a defined and explicit way [10]. It should be remembered that much text is primarily intended for visual consumption, and aspects ranging from the typography through to the vocabulary are chosen to aid comprehension. For example, both italic and bold fonts are used for

emphasis. By the time the text is in electronic form, in particular ASCII, much of this visual presentation has already been lost, although an increasing number of texts are created specifically for an electronic medium.

This stage also requires a limited amount of text generation, e.g. when converting a digit string into orthographic words. This can cause some significant problems, because there may not be enough information available at this stage to decide exactly what the generated text should be. For example, in many Romance languages, a number must agree in gender with its 'object', and it may not be clear what the object is without a full syntactic parse.

The process

The input text will have many of the following features:

- embedded control characters,
- symbols,
- indentation,
- white space,
- capitalisation,
- numerals,
- punctuation,
- abbreviations.

The normalised form is typically a sequence of explicitly separated words, consisting just of lower case letters, with punctuation associated with some words, and enough other information to correctly 'read' the original text. An example is shown in Fig 3.

In Fig 3 the normalised words appear towards the bottom of the structure, with the preceding and following punctuation immediately above and below respectively. Words which had an initial capital in the input text are marked 'CAP', numerals with 'NUM', and abbreviations with 'ABBR'.

Assuming that segmentation has been applied to sentence level, the most straightforward approach is to scan through the sentence using white space to separate out words. Some characters and strings will be converted during this process, dependent on their context and on which optional conversions have been set by the application. The following sections describe a typical set of conversions for an English TTS system.

Control characters and white space

Every control character, including the 'newline' character is converted to a space. This assumes that enough

input: "It's 206km. to John's 'home' of St. Andrews" said Rev. Jim.

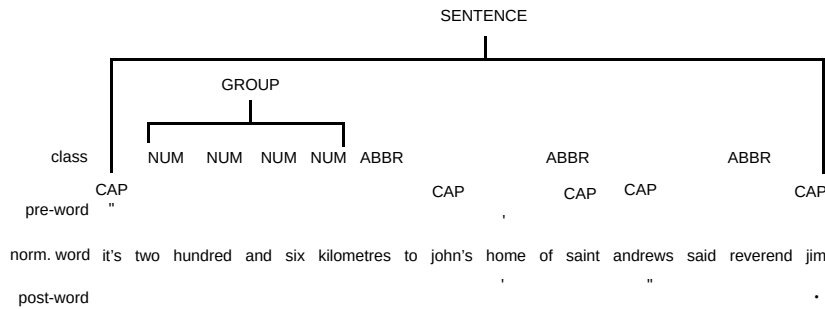


Fig 3 Example sentence and its normalised representation.

structural information, i.e. sentence and paragraph determination, has already been found in the segmentation stage. For example, newline characters are used to determine sentence boundaries.

Capitals

Capitalisation normally occurs in five situations:

- the first letter of the first word in a sentence,
- proper nouns (all nouns have an initial capital in German)
- acronyms or abbreviations,
- initials, e.g. J. M. Smith,
- emphasised text, including section headings.

Abbreviations can be sensitive to upper-case letters, but otherwise capitalisation cannot be relied upon as a precise indicator of a word, although it will bias later decisions, such as whether special dictionaries are searched. Therefore all words are converted to lower case, and capitalisation information is retained.

Acronyms are identified as words where any non-initial letter is a capital, but if all letters are capitals, it could just be an emphatic word. Emphatic words tend to occur together, either as a phrase, or as a complete sentence (such as headings), so the form of surrounding words can bias the decision.

Numerals

Numerals occur frequently in text, and are used in a number of different ways, e.g as ordinals, dates, times, currencies, or in ‘special’ formats such as telephone numbers. The digit sequence is replaced by one or more words, according to the digit pattern.

Any simple digit strings, possibly including a full stop or comma, and matching none of the other patterns below, are treated as a default number and are converted to the

corresponding words, e.g. '1245.67' becomes 'one thousand two hundred and forty five point six seven'. Ordinals are an integer followed by 'th', 'nd' or 'rd', e.g. '4th', '123rd'. The conversion is the same as a default number, but with the last word changed to its ordinal form, e.g. '123' becomes 'one hundred and twenty three', but '123rd' becomes 'one hundred and twenty third'.

Dates and times

Dates are very common in texts and occur in many formats, for example, '12-8-94', '12/8/94', '12.08.94', '12-Aug-94', and '12-aug-1994' should all be read as 'twelfth of august nineteen ninety four'. Most of these patterns should also be recognised as dates even without the year specified. The first three patterns have a different interpretation under the conventions of American English, where the standard date format is Month-Day-Year. This can clearly be automatically determined if the date were only meaningful using one convention, e.g. '13-10-94'. It is also useful to recognise a partly normalised date format, e.g. '12th December 1994', since the last digit string should be read as 'nineteen ninety four', rather than the default number 'one thousand nine hundred and ninety four'.

Times are defined by the patterns HH:MM or HH:MM:SS, where HH, MM and SS represent hours, minutes and seconds respectively. Range checking is performed on each of these values. If the 12-hour clock is used, the abbreviations am or pm may follow the time.

Currency

Currency formats start with a currency symbol, which may be a letter sequence, followed by a number, optionally followed by an amount abbreviation, e.g. ‘#12m’, ‘DM234bn’, ‘\$32’. If there is no amount abbreviation and the number has two digits following the point, then the interpretation is currency and sub-currency, e.g. ‘\$12.34’ becomes ‘twelve dollars and thirty four cents’, but ‘\$12.3m’ becomes ‘twelve point three million dollars’.

Grouping

When a 'unit' in the original text is split into many normalised words, e.g. when expanding a number, it is important to remember that the words are grouped together. Any further structure of the grouping should also be stored, e.g. code and number format of telephone numbers, or parts of credit card numbers, etc. This information, along with the pattern type, can be used by the prosodic generation components later in the system.

Abbreviations

Abbreviations, which may end with a full stop, are normally expanded to their full value (as determined by a table supplied by the application), but there are some cases which require special treatment.

- Ambiguous abbreviations commonly occur in addresses, and can expand to more than one word depending on context. Common examples are St. which can expand to saint or street and Dr. which can expand to doctor or drive. A simple positional dependence can usually determine which word to expand to, but there are still cases that are hard to distinguish; for example, consider the following pair: 'Mary St Mead' should become 'mary saint mead' but 'London St North' should be read as 'london street north'.
- Homographic abbreviations can also appear as words e.g. 'sat' (saturday), 'in' (inch), 'is' (island), etc. This can cause a problem in sentence segmentation if the last word in the sentence is a homographic abbreviation (see example g above in section 2.2). The most reliable solution is to look for secondary cues to sentence termination, such as a blank line or initial capital on the following word. As there will be cases where the ambiguity cannot be resolved, a default expansion is used.
- Measurement abbreviations occur after numbers, and the expanded abbreviation must agree with the number, e.g. '1 cm' becomes 'one centimetre' but '7 cm' becomes 'seven centimetres'.

Punctuation

All punctuation is normally associated with the nearest word, and is stored as either pre-word or post-word punctuation. However, certain punctuation marks have a default binding which overrides this, e.g. open brackets (or upside-down question mark in Spanish) bind to the following word. It should be noted that it is perfectly possible for more than one punctuation mark to be associated with a word, especially when quotations occur, e.g. He said, "I saw John.", or multiple sentence terminators are used (Because... or Hello!!!). A single quote

might not be punctuation, but part of the word itself, either internally in the case of contractions and possessives (didn't, John's), or externally in plural possessives (dogs' bowl) and heavy orthographic reductions ('tween or roamin').

Symbols

Various symbols are defined in ASCII which are not part of normal written text, such as curly brackets, hash, tilde, etc. These will be converted to the symbol name, which may depend on the application. Some symbols may already have been dealt with by earlier conversions e.g. '\$' in a currency format.

3. Morphology and pronunciation

The text segmentation and normalisation task provides a stream of separate words from the original text. The morphology and pronunciation task analyses every word to determine its possible syntactic functions and pronunciation. This process may produce alternative hypotheses for words or word sequences; it is the role of the tagger and parser (see section 4) to generate the correct unambiguous sequence.

3.1 Morphology

The morphological component is closely tied to the pronunciation component. It has three functions:

- grouping words into lexical phrases,
- marking the lexical syntactic information, i.e. part of speech for each word,
- generating the root-form, or stem morphology, of each word for subsequent pronunciation.

Lexical phrases

The word-tokenisation used in the normalisation component assumes that white space separates the text into distinct words. However, it is often more natural to treat certain word sequences as a single lexical entry, either because of a variant pronunciation, or more commonly, because the lexical phrase acts as a single unit with a single syntactic category. In most cases lexical phrases are idioms or phrasal verbs which are more naturally treated as single units. For example, the phrase 'up and over' can act as if it were a single adjective, as in the sentence:

It had an up and over door.

but it is important to also store the individual words, since this sequence of words may be used in another sense:

He climbed up and over the hill.

Lemmatisation

For efficiency, and to avoid duplication, the lexicon is often lemmatised (reduced to headword entries). This achieves a significant reduction in space. For example, considering the word 'love'; in a full-form dictionary it would be necessary to store the words: love, loves, lover, lovers, loved, loving, lovingly, unloved, unloving, unlovingly, etc, whereas a lemmatised dictionary would just store the root-form, or headword, 'love' and an indication of the appropriate affixation rule sets for deriving all the other surface forms. This leaves the problem of deriving the affixation rules, but there is abundant literature on this subject, e.g. Choumsky and Halle [11], Gimson [12], Allen et al [13]. Rather than store a reference to every affix rule in an entry, rules can be grouped into overlapping rule sets, and a reference to allowed rule sets is stored in the entry. It should be remembered that, as this part of a TTS system is only used to analyse existing words, not to generate them, many of the complexities introduced to prevent over-generation of surface forms are unnecessary. Since this paper considers practical systems (not just theoretically rigorous research systems) irregular surface forms can always be stored directly in the dictionary as a new headword.

Affixation and compounding

The morphological component can now use a lemmatised dictionary to search for any given word in the input text. This is a fairly easy task in theory, but there are some implementation issues to consider here. The lemmatised dictionary described above is effectively indexed on headword, not on surface form, but to analyse an incoming word, only the surface form is known. There are a number of possible solutions, which are described in the following paragraphs.

Since most affixation is actually suffix adding, the dictionary could be directly indexed on the fixed part of each headword, and the affixation applied, if necessary. There must either be some process to check for prefixes, or simply disallow prefixes, and add the extra words as new headwords. This solution is clearly not applicable for languages that make extensive use of prefixes.

An index could be created of all full form words, but this would defeat the object of lemmatising the dictionary in the first place.

A recursive procedure can be used which tries any affix rule matching the current word, then attempt to find the resultant word as a headword. This can perform several recursions. If a match is found, it is necessary to check that the affix rule(s) used was valid for the matched headword. As an example, consider the case where there are two headwords, 'star' and 'stare', and two affix rules, '+ing'

(which adds 'ing' to the root) and '-e+ing' (which removes the last 'e' from the stem then adds 'ing'). If the incoming word is 'staring', there are two possible analyses: 'star+ing' or 'stare-e+ing' (see Fig 4).

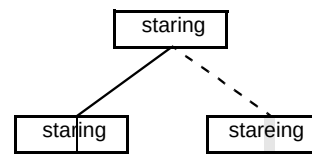


Fig 4 Two analyses of the word 'staring'.

Clearly the first analysis is incorrect (adding 'ing' to 'star' would require consonant doubling, giving 'starring'). Therefore it is necessary to constrain the allowed affix rules, which in this case would mean only allowing the '-e+ing' rule to apply to the 'stare' entry.

A modification of the last approach uses a chained list to connect all entries which may use a particular rule. When a rule is activated, only entries on its chain are active, and can be searched. This way the forward search is constrained, rather than the backward constraint in the previous method.

The compounding process applies in a similar way, using the compounding rules appropriate for each entry.

Lexical syntax

Every word in the lexicon will be marked with at least one syntactic category, or 'part of speech'. Many words have more than one syntactic category, for example 'man' and 'stall' can both act as verbs or nouns -- so 'man the stall!' and 'stall the man!' are both reasonable sentences. The detail encoded in the syntactic category varies greatly between systems, but it will at least enable the system to distinguish between major parts of speech such as verbs, nouns, adjectives, adverbs, pronouns, etc. The entry may encode far more detailed syntactic characteristics of the word, such as a verb's transitivity, and this information can enable more sophisticated processing in later components. The main use of the syntactic category information is for the parser to build up the sentence structure, and if the parser includes a stochastic tagging component (see section 4 below), it is beneficial for the lexicon to include the relative frequency of each possible category for a word.

If the incoming word does not appear in the lexicon, even after a morphological analysis, it is still necessary for the system to have some syntactic information about the word. The result of the morphological analysis itself will often give a strong indication of the word's category, as many suffixes indicate a particular part of speech (e.g. words ending in 'iness' are very likely to be nouns). This characteristic leads to a robust method for determining the syntactic category of non-lexical words, since a table can be built giving the cumulative corpus category statistics for words ending in any given letter sequence.

3.2 Pronunciation

TTS systems traditionally used a set of letter-to-sound rules to produce pronunciations, and only resorted to a dictionary to hold exceptions to these rules. However, the current practice is to use a large on-line pronunciation dictionary, and revert to rule-based methods as a fall-back approach for non-lexicon words. There are three reasons for this change: increase in overall quality has made pronunciation errors more significant, the search time for a dictionary is often quicker than generation by rule, and, most significantly, a large dictionary is already needed to store other lexical information such as syntax, lexical semantics, and morphological information.

Representation of a pronunciation

Typically a pronunciation will be given to every word (or lexical phrase) fairly early on in the text analysis stage. Some later process will consider the effect of syntactic structure and of the contextual effects of surrounding words. This would imply that the pronunciation task should produce an 'archetypal' transcription for every word, often referred to as the 'citation form'. This is defined as the pronunciation given to a word when it is produced in isolation, rather than connected speech.

The issue of how best to represent phonemes, allophones¹, stress and syllabification information for a pronunciation has been addressed many times within the speech community. The International Phonetic Association (IPA) publishes probably the most widely accepted symbol set, but it is not particularly suitable for computational systems, since it consists of a mixture of Roman, Greek and other miscellaneous symbols, not represented in a standard computer character set. Furthermore, the IPA alphabet is not 'theory-neutral' (it is motivated by a theory of word-phonology) so it uses a discrete cross-parametric segmental representation, where symbols are chosen mainly by their similarity to standard orthographic representations, not to any characteristics of the sounds themselves. The differing requirements of speech researchers has spawned a plethora of machine readable symbol sets [4, 14--16], which tend to have different coverage and scopes. Example pronunciations in this paper are shown in SAMPA symbols for British English, shown in Table 1.

It is fairly clear from the discussion above, that there is no 'correct' way to represent even the agreed citation form of a word, since the selection of exactly what is represented is subject to a theoretical bias. Having made this caveat, there are, broadly speaking, two types of transcription:

- phonemic, which attempts to detail the distinctive contrastive sound units used in the accent,
- phonetic, which details the actual sounds used in the production.

Table 1 SAMPA symbols for British English.

p	<u>p</u> ear	t	<u>t</u> ear	k	<u>k</u> ing
b	<u>b</u> ear	d	<u>d</u> ear	g	<u>g</u> ear
f	<u>f</u> ear	T	<u>th</u> ing	s	<u>s</u> ing
v	<u>v</u> ery	D	<u>th</u> is	z	<u>z</u> ing
S	<u>sh</u> eer	l	<u>l</u> eer	j	<u>y</u> ear
Z	treas <u>u</u> re	w	<u>w</u> ear	r	<u>r</u> ear
m	<u>m</u> en	n	<u>n</u> ear	N	w <u>in</u> g
tS	<u>ch</u> eer	=n	butt <u>o</u> n	h	<u>h</u> ear
dZ	<u>j</u> eer	=l	bott <u>l</u> e		
@	<u>a</u> go	{	bat	E	<u>b</u> et
I	<u>b</u> it	Q	<u>c</u> od	U	<u>g</u> ood
V	<u>b</u> ud	3	bird	A	<u>b</u> ard
i	<u>b</u> ead	O	bore	u	<u>b</u> oot
@U	zer <u>o</u>	aI	<u>pie</u>	aU	<u>cow</u>
E@	hair	eI	<u>pay</u>	I@	<u>peer</u>
OI	<u>boy</u>	U@	<u>Ruhr</u>		
"	Primary stress		\$	Syllable boundary	
'	Secondary stress		#	Word boundary	

To a native speaker, who is at least subconsciously aware of how sound units are realised in their language, these two transcription types may seem artificially separated. However, it should be remembered that the phonemic description is based on an implicit analysis of the meaningful contrastive sounds in a particular language and accent, at a particular time in history and only through this analysis can any claims be made about the actual production of speech based on the phonemic transcription.

For the purposes of synthesis, it is also necessary to mark lexical stress (or word stress) in the pronunciation. This gives rise to contrastive pronunciations of words which have essentially the same segmental description, e.g. compare the word 'increase' in the two phrases 'increase the pay' and 'pay the increase'. Lexical stress may be marked on a whole syllable, or in a non-syllabic analysis marked only on the nuclear element, which is invariably a vowel. There are many proposed systems of lexical stress [12] — using hierarchical stress, binary stress, ternary stress (primary, secondary and unstressed) or even four levels of stress. Some analyses further distinguish between the different realisations which stress may have in the speech signal, contrasting secondary stress which may take a pitch accent, and that which cannot.

¹ An allophone is a particular variant pronunciation of a phoneme, which does not change the meaning of a word, e.g. the final/p/ in 'stop' may be realised with or without a strong breath sound, but most listeners do not distinguish the pronunciations.

Coverage of a lexicon

Since it is rarely feasible to generate an exhaustive list of all words in a language, one question that must be addressed in the design of a TTS system is how big the lexicon should be. There is no simple answer of course, but there are common trends in word distribution of which use can be made. Figure 5 shows the frequency of each of the 13 334 unique words in a newspaper corpus, C1, of 108 000 words [17]. It can be clearly seen that about half of all words only occur once in the corpus, and 90% occur less than 10 times. Many of these rare words are proper nouns, such as surnames and place names. The same data is presented as a graph of top-n words against corpus coverage in Fig 6.

Figure 6 shows that the top 1% of words make up about 50% of the corpus, the top 10% of words make up 75% and the top 40% make up 90% of the corpus. Clearly doubling the lexicon size from 1000 to 2000 words gives more coverage than doubling from 4000 to 8000 words. A very similar pattern can be seen in Fig 7, which gives the word distribution for another similar sized newspaper corpus, C2, of 89 000 words, of which 11 377 are unique. Figure 7 also shows that the words from C1 only provide a partial coverage of C2 (87%), even though C1 is larger. It is interesting to note that the first 1000 words from C1 or C2 provide approximately the same coverage of C2. In fact, the top 1000 words from C1 and C2 are almost identical, and will be similar in most texts; it is the low-frequency words which differ significantly between texts.

The observations above indicate that the lexicon should include the top few thousand words, but beyond about 5–10 000 the gain becomes quite small, and increasing the lexicon

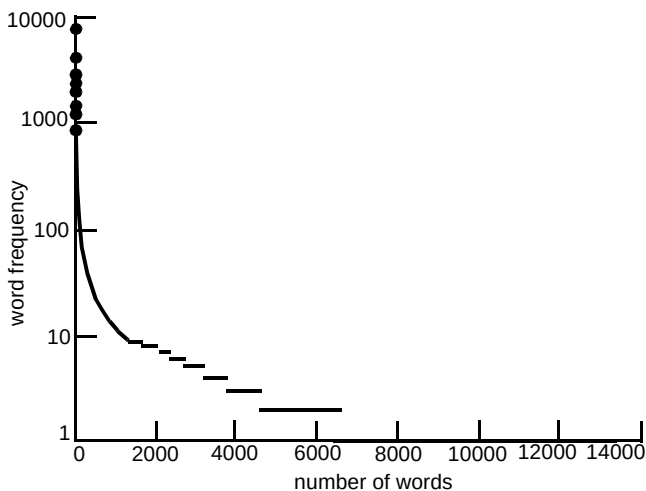


Fig 5 Word frequency for corpus C1.

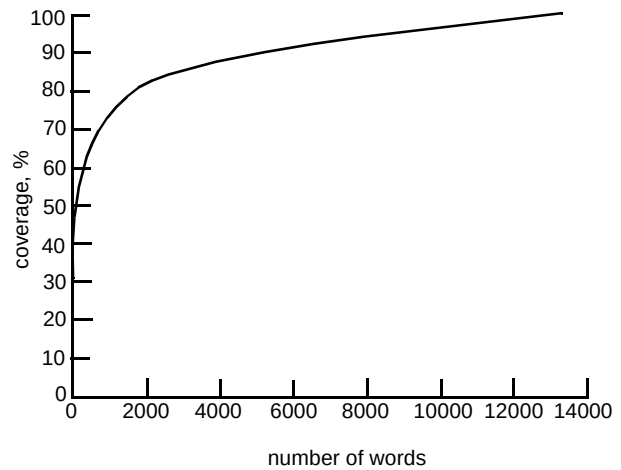


Fig 6 Top-n word coverage for corpus C1. [Image]

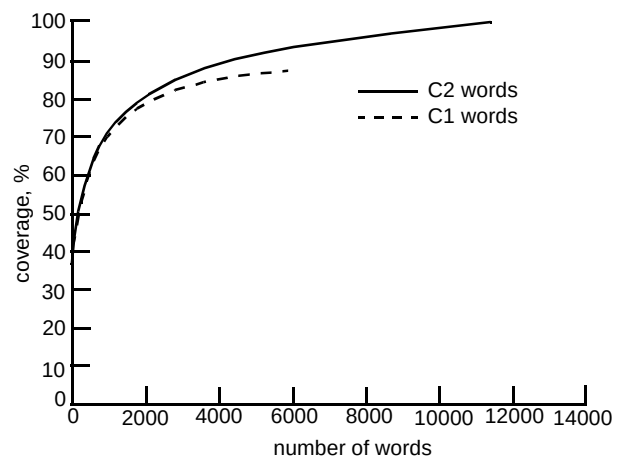


Fig 7 Coverage of corpus C2 by Top-n words from C1 and C2.

size will increase search times. The final size of the lexicon will also be affected by the accuracy and reliability of other fall-back pronunciation methods.

Fall-back methods

When a word is not found in the lexicon, it is necessary to generate its pronunciation by another method. The most common approach is to use letter-to-sound rules, which convert from the spelling of a word to the pronunciation. For some languages, such as English, this is a difficult process, as with the famous 'ough' words: through, though, rough and bough, where the 'ough' part is pronounced differently for each word.

The main emphasis in the past decade has been on increasing dictionary size, and the area of unknown word pronunciation has been relatively neglected. Klatt [4] provides a good description of non-lexical pronunciation methods for English, in which he describes the evolution

and application of letter-to-sound rule techniques. Klatt also describes the early development of pronunciation-by-analogy techniques. These are based on the theory that when confronted with an unknown word, a human reader will pronounce it like any similar names they know. For example the name Plotsky would be pronounced as /'p l Q t \$ s k i/ to rhyme with Trotsky /'t r Q t \$ s k i/.

Proper names

There has been much interest in the pronunciation of proper names, mainly due to the many applications which require pronunciation of names and addresses from large customer databases [18]. Names often have a 'fossilised' pronunciation with respect to general words in a language since they are owned by individuals and are therefore more resistant to the gradual process of language change. Names are also continually moving around between languages and cultures with their owners, and may take several generations to assimilate into a new 'nativised' pronunciation. These features make pronunciation-by-analogy a good candidate method for names pronunciation [10].

Although there have been attempts to produce large name pronunciation dictionaries, in particular the European ONOMASTICA project [19], their coverage is still too small to make non-lexical methods redundant.

Spelling

The final fall-back method, if all other approaches have failed, is to spell the word. This seems a very natural thing to do, but it actually takes a while for a listener to realise that a spelling sequence is occurring if it comes in the middle of continuous speech, and is not 'sign-posted' in some way. If a difficult word appears when a human is reading, the speaker will normally attempt a pronunciation, then signal a switch to spelling mode with the cue 'which is spelt as...' or 'spelt...', then spell the word in question.

The spelt sequence should be marked in such a way that later stages of the TTS system know that it is a spelt sequence, and came from a single word. This eases the job of the parser, which can treat the sequence as a single unit, and allows an appropriate prosody to be imposed on the spelt sequence.

Having generated the pronunciation(s) and possible syntactic categories for each word in the input text, and dealt with any unusual cases, the next task is to decide how the words relate to each other, i.e. determining the syntactic structure of the sentence.

4. Syntactic analysis

The syntactic analysis task takes the sequence of words in a sentence and, from the lexical syntax, derives a structural analysis for the sentence. This is an important task for text-to-speech systems, in particular for prosodic generation [5], which can make use of the sentence structure to derive the sentence's prosodic structure. However, since language is so inherently ambiguous, knowing the structure of the sentence does not necessarily equate to knowing the meaning of the sentence. For example, consider the simple sentence:

If the display is not bright, change the batteries.

Ignoring context for the moment, 'display' and 'change' could be verbs or nouns, and 'bright' could be an adjective or adverb, giving 8 possible part-of-speech (or tag) sequences for the sentence. However, an analysis of the possible meanings of the sentence based on the product of the meanings of each word [20], gives 6116 semantic interpretations of the sentence. If the correct part-of-speech sequence and syntactic structure is known, this number only reduces to 1764. Luckily, it is not necessary for a system to understand the sentence to make a reasonable attempt to read it; the system only needs a knowledge of the structural relationship between parts of the sentence. Of course, understanding the sentence would improve a system's reading ability.

Knowing the syntactic category of every word may also be important to determine pronunciation, for example:

He was *content* (adjective) with his lot.

The *content* (noun) was not exciting.

The *convict* (noun) escaped.

You *convict* (verb) him.

The first job of the parser component is to assign such syntactic categories to each word. The input to this task is a list of possible categories for the words -- these categories coming from lexical sources, or if no lexical source exists, coming from morphological analyses of the structure of the word (see section 3.1).

4.1 Stochastic tagging

The most common method for tackling this task is the stochastic tagger. This uses a statistical model of language, which assigns words their most likely part of speech, given the probability of the words appearing in a certain context, and also the probability of the tags appearing on the words themselves. The model is derived from large corpora of correctly tagged text [21].

The probability that a particular sequence of words, W are tagged with a particular tag sequence, T is represented by the conditional probability $P(T|W)$. The conditional probability $P(T|W)$ can be expressed in Bayes theorem as:

$$P(T|W) = \frac{P(W|T)P(T)}{P(W)}$$

where the important parts are the probability that specific words are generated given specific tags $P(W|T)$, and the probability that the tags occur in that particular sequence $P(T)$. Note that, since W is the same for all hypothesised tag sequences, $P(W)$ can be disregarded. Obviously certain tag sequences are better than others, so the aim is to maximise $P(T|W)$.

The probability of each sequence can be rewritten as a product of the conditional probabilities of each word or tag, given all of the previous tags:

$$P(T|W)P(W) = \prod p(t_0)p(t_i|t_{i-1}, t_{i-2}, \dots) p(w_i|t_i, \dots, w_{i-1}, \dots).$$

Now, the approximation can be made that each tag depends only on the immediately preceding tags (n -gram model), and that the word depends only on the tag. For a trigram model there would then be, for example:

$$P(T|W)P(W) = p(t_0) \prod p(t_i | t_{i-1}, t_{i-2}) p(w_i | t_i)$$

Once a tag is known to have been selected, no further information is gained from knowing the previous tags or words; so the above model is a Markov model.

With training data n -gram sequence probabilities and the probability of each word being given a tag (lexical probabilities) can be estimated. However, the model must also cope with words that do not appear in the lexicon. For this, information from a morphological decomposition can be used to provide estimated probabilities based on suffixes and capitalisation (see section 3.1).

Given these probabilities, the most likely tag sequence for a given word sequence can be found. One way would be to simply compute all possible tag sequences and find the maximum, but this would require an exhaustive search (although given that some words only have certain tags, the search space is reduced somewhat). Fortunately, standard algorithms, such as Viterbi [22] allow us to find the most likely tag sequence efficiently.

Given a sequence of tags for a particular language, although there are long distance dependencies between words far away in the written sentence, the probability of a tag occurring can be adequately determined by viewing a tag sequence history of 2 words. Such bi-gram taggers attain a useful level of accuracy, of about 95--96% on unseen text [21] (i.e. text on which they have not been trained). It is

interesting to note that if the most likely tag is chosen for a particular word (the tag occurring most often for that word in a tagged corpus), accuracy is often about 92--93%.

Using higher level n -gram taggers does not seem to increase this figure enough to justify the investment in obtaining suitably large training corpora, nor the computational increase involved in their implementation. It is unlikely that current pure stochastic taggers can be improved on, since at the word sequence level, certain syntactic and semantic ambiguities arise which cannot be resolved without recourse to other levels of analysis. Fortunately, for pronunciation purposes, the number of words having multiple pronunciations is quite small -- between 1 and 2% of words in a typical lexicon. In practice there is less than a 0.3% chance of picking the wrong pronunciation for a word.

The robustness and speed of the stochastic taggers mean they are generally favoured over rule-based taggers in current natural language processing systems, and also provide an appropriate solution for commercial text-to-speech systems.

Rule-based taggers generally do not exist as such, but tend to form part of the higher level syntactic parsing. The purpose of this task is to assign a structure to the sentence which reflects the syntactic role of every word in it. By constructing a syntactic structure which considers the role of each word in the sentence, decisions can be made on individual word syntactic categories. Many rule-based parsers, however, use stochastic taggers as front ends, and parse with the tags given.

4.2 Parsing

The main approach favoured by syntactic parsers is the rule-based approach, where grammar rules, which may be recursive, describe valid sequences of symbols. These symbols correspond to word tags, or groups of words representing phrases, clauses or even whole sentences. There are many linguistic theories and corresponding rule-based grammars, but the fact that no one grammar formalism has come to the fore is an indictment of the state of the art with regard to rule-based parsing.

Statistical parsers exist which attempt to assign structures above the word level, by training on corpora of full syntactic structures [23]. The general lack of large syntactically analysed corpora have so far limited this approach, but perhaps a more fundamental problem is that the infinite theoretical structures possible in language limit the statistical approach -- some generalisations must be made. There are several innovative approaches which incorporate probabilities into the application of grammar rules themselves [24], and these claim a 60% accuracy. Rule-based syntactic structure parsers can claim much

higher accuracy than this, but they suffer from massive over-generation of incorrect structures, which limits their usefulness. Indeed stochastic parsers can also be viewed as suffering from over-generation, but, by their very nature, they impose a rank ordering, and often return only the most likely structure.

Ambiguity

The reason for this massive over-generation boils down to two main sources — lexical and structural ambiguity. Lexical ambiguity, where words have more than one sense, was demonstrated at the start of this section.

Structural ambiguity is where there is more than one possible analysis of the attachment of words in the sentence. The two are not mutually exclusive. There are three main classes of structural ambiguity.

- Prepositional phrase attachment — “The man saw the boy with a telescope.”

where the telescope may have been used to observe the boy (Fig 8 (a)), or the boy was carrying a telescope (Fig 8 (b)). When more prepositional phrases are added, the number of ambiguities increases, e.g. ‘The man saw the boy in the park with the telescope.’

- Noun-noun co-ordination — ‘Pressure cooker balance weight adjustment screw.’

[[[pressure cooker][balance weight] [adjustment screw]]

It is fairly easy for humans to identify boundaries between noun compounds, but this task is extremely difficult for a machine.

- Co-ordination — ‘The old men and women.’

where the analysis is either ‘old men’ and ‘women’, or old ‘men and women’.

Semantic restrictions, or world and contextual knowledge may be used to filter clearly incorrect structures, but, as has been the case with AI systems facing the same problem, these methods have only been proven in systems of limited domain. Therefore, it is not uncommon for parsers to return many thousands of possible analyses for even small sentences, only one of which is the correct one! This of course has performance implications in the number of nodes generated in a parse tree.

Efficient parsing techniques such as chart parsing [25], extended by packing and indexing of edges, have been successful in addressing the generation of multiple analyses for one sentence.

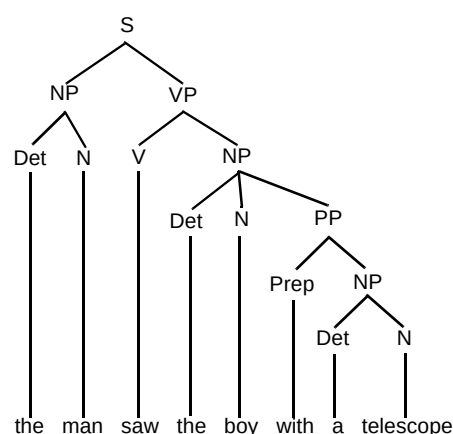
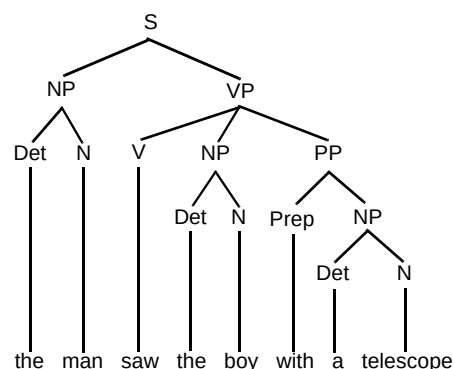


Fig 8 Prepositional phrase structural ambiguity.

The over-generation of multiple structures is related to the granularity and recursiveness of the grammar rules. No complete computational grammar of English exists, despite years of research, and probably never will. Human language is constantly evolving and expanding, and the notion of grammaticality is a theoretic device that does not always relate to how people actually use language.

One area of language evolution is idiomatic expressions, but by changing the grammar rules to accommodate these exceptions, rules are often introduced which cause more structures to be generated — over-generation. At the other extreme, using fewer rules leads to the risk of the parser failing to find an analysis for what might be considered an acceptable sentence — under-generation. Another possibility is to use fewer rules to recognise islands of partial linguistic structures, and ignore the attachment problems, except when confident attachment predictions can be made.

By combining the best of both worlds (stochastic word tagging and syntactic structure parsing with a partial grammar), it is possible to derive a fast and efficient parser, albeit at the expense of a rich syntactic analysis. An important aspect of this approach is that ill-formed input

can be accommodated, an important feature for TTS. An example of such an analysis is shown in Fig 9.

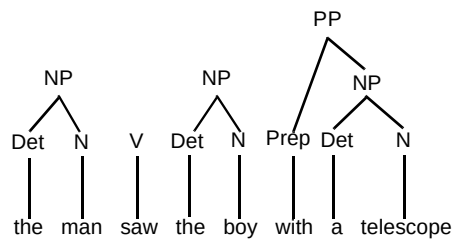


Fig 9 Partial parsing.

This is a more sparse structure than in Fig 8, but it provides a useful level of information for input to the performance parser. Current research would seem to be focused both on hybrid approaches selecting the best features of rule-based and stochastic parsers — for example, probabilistic parsing with grammar rules, with back-off if no parse can be found — and on incorporating novel techniques for treatment of idioms and collocational phrases which make up so much of normal language use.

5. Post-lexical processes and continuous speech effects

In the description in section 2.5, the process of pronunciation and stress alignment work at the word level, i.e. they work on information which is described in the lexicon. However, there are a host of post-lexical processes that function across word boundaries. Lexical phonology for example divides the phonological analysis of language into two parts, a lexical component and a post-lexical component (sometimes referred to as phrasal phonology). This post-lexical phase may be further subdivided into processes which are phonologically motivated, and processes which are phonetically motivated. However a clear boundary between the two subdivisions is difficult to derive.

The pronunciation task may be viewed as functioning like the lexical component described above, while the continuous-speech-effects component works at the level of the post-lexical component. Goldsmith [26] states that:

“The post-lexical phonology involves two major sorts of rule applications: (i) those operating crucially across word-boundaries or making specific use of phrasal or syntactic structure, and (ii) those that fill in, specify, or refer to non-distinctive features.”

Goldsmith’s rules of type (i) change a word’s pronunciation due to its phonological and syntactic context. These changes are very common in normal speech, and are

necessary for natural-sounding speech synthesis. Rules of type (ii) are applied primarily according to the style of speech (e.g. speech rate) and would be used by a TTS system to model more subtle effects of fluid speech, requiring careful application to avoid compromising intelligibility. Examples of both types of rule in a TTS system are discussed below.

There is, in spoken English, a tendency for words to be pronounced differently depending on their position within the syntactic structure of a sentence. As a result, most words are said to have a strong and a weak form. Some words, however, are more prone to this form of reduction than others. These words, are often called function or grammatical words, e.g. auxiliary verbs, prepositions, pronouns and conjunctions, etc. For example, the determiner ‘the’ has weak forms /D@/ as in ‘the man’ /D@ m{n/ when spoken before a word beginning with a consonant and /Di/ as in ‘the apple’ /Di {p=l/ when spoken before a word beginning with a vowel. The words ‘a’ and ‘an’ have the weak forms /@/ as in ‘a bag’ /@ b{g/ when spoken before a word beginning with a consonant and /@n/ as in ‘an apple’ /@n {p=l/ when spoken before a word beginning with a vowel. Notice how the surface pronunciation of the weak form is dependent upon the underlying lexical pronunciation of the word and the phonemic context.

Without such reductions continuous speech sounds stilted and unnatural, while over-application or inappropriate application of such post-lexical rules can reduce the intelligibility of the synthetic speech.

The phonological rules described above are predominantly driven by the grammatical structure of the utterance. There are a class of post-lexical processes whose application or degree of application is dictated by speech style. The domain of application of such rules is across word boundaries. These processes include elision, assimilation and r-insertion.

Elision is the processes where under certain circumstances, a phoneme may be deleted or more accurately have zero realisation. Elision can act both within words and across words at word boundaries. Within-word elision affects the weak syllables of a word, for example the word ‘perhaps’ may be realised as [ph’’h{ps]¹ where the unstressed vowel has been elided and the aspiration of the initial plosive has taken up the whole of the middle portion of the syllable. Elision often affects clusters of complex consonants such as ‘looked back’, where /lUkdb{k/ may be pronounced /lUkb{k/.

¹ This transcription is phonetic: the p^h notation indicates an aspirated /p/, i.e. there is a strong breath sound following the release of the /p/.

As assimilation affects consonants at word boundaries. For example take the phrase 'that colour'. The citation form pronunciation of these words is /D{t/ and /kVl@/. Yet when produced as a phrase in casual speech, the process of assimilation changes the production of the consonants at the word boundaries so that the phrase is pronounced as /D{kVl@/. Elision and assimilation may act repeatedly on a phrase, for example the phrase 'hand bag' may be pronounced /h{nd bag/ in over articulated speech, or as /h{nbag/ or even /h{mbag/ in casual speech. As with the production of weak forms, appropriate application of these processes increases the naturalness of synthetic speech.

R-insertion, like assimilation, is clearly a post-lexical process as it acts across word boundaries. The phonological rule for r-insertion can be simply stated as:

Zero $\rightarrow$ /r/ [@, 3, l@, E@, A, O] # V

This states that the phoneme /r/ can be inserted between a word ending in one of the vowels contained within the brackets and a word starting with any vowel, e.g. 'for Arthur' /fO r AT@/.

Recently a number of researchers have suggested that many post-lexical processes which traditionally have been described in terms of boundaries and syntax would be better represented under some form of prosodic hierarchy. Durand [27] cites the example of the domain of application of [r]-insertion and elision as a strong argument in favour of using a prosodic hierarchy.

6. Conclusions

The process of converting text into speech has been presented as a two-stage process, text and linguistic analysis, followed by prosody and speech generation. This paper has described techniques used in current state-of-the-art commercial TTS systems to perform the analysis stage. In particular, the variability and variety of input text was illustrated, and approaches were described to robustly extract and formalise the information encoded in the input text for later processing. Linguistic analysis could then proceed, starting with morphological analysis, which examines the structure of each word, aiding the pronunciation generation task. Syntactic analysis then derived a structural analysis for phrases, clauses and/or sentences. This must work robustly, providing the most appropriate analysis for subsequent prosodic generation. Syntactic information is also needed in order to model a number of continuous speech effects, which give the speech more fluent and natural characteristics.

The goal of this analysis is to extract all the information provided by the text deemed relevant for speech synthesis. The paper covering the second part of this subject [5] describes the process of generating an appropriate prosody

for the synthetic speech, and discusses techniques used in speech concatenation and modification to create the final signal.

Acknowledgements

The authors wish to thank a number of colleagues for useful discussions and help in formulating many of the ideas in this paper. Special thanks go to Maggie Gaved, whose work provided the basis for much of the text normalisation section, and to other colleagues in the speech synthesis and analysis group at BT Laboratories.

References

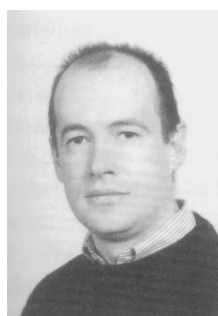
- 1 Page J H and Breen A P: 'The Laureate text-to-speech system — architecture and applications', *BT Technol J*, **14**, No 1, pp 57—67 (January 1996).
- 2 Johnston R D: 'Beyond intelligibility — the performance of text-to-speech synthesisers', *BT Technol J*, **14**, No 1, pp 100—111 (January 1996).
- 3 Moulines E and Charpentier F: 'Pitch-synchronous waveform processing techniques for text-to-speech synthesis using diphones', *Speech Comm*, **9**, No 5/6, pp 453—467 (December 1990).
- 4 Klatt D: 'Review of text-to-speech conversion for English', *J Acoust Soc Am*, **82**, No 3, pp 737—793 (September 1987).
- 5 Edgington M et al: 'Overview of current text-to-speech techniques: Part II — prosody and speech synthesis', *BT Technol J*, **14**, No 1, pp 84—89 (January 1996).
- 6 Lindstrom A and Ljungqvist M: 'Text processing within a speech synthesis system', *Proc ICSLP '94*, Yokohama, pp 1683—1686 (September 1994).
- 7 Jones B: 'Exploring the variety and use of punctuation', *Proc 17th Annual Cognitive Science Conference*, Pittsburgh (July 1995).
- 8 Carvalho P et al: 'E-mail to voice-mail conversion using a Portuguese text-to-speech system', *Proc ICSLP '94*, Yokohama, pp 1687—1691 (September 1994).
- 9 Silverman K: 'The structure and processing of fundamental frequency contours', PhD These, University of Cambridge (1987).
- 10 Gaved M: 'Pronunciation and text normalisation in applied text-to-speech systems', *Proc Eurospeech '93*, Berlin, pp 897—900 (September 1993).
- 11 Chomsky N and Halle M: 'The sound pattern of English', MIT Press (1968).
- 12 Gimson A C: 'An introduction to the pronunciation of English', 4th Edition, Edward Arnold, London (1989).
- 13 Allen J, Hunnicutt W S and Klatt D: 'From text-to-speech: the MITalk system', Cambridge University Press (1987).
- 14 Seneff S and Zue V W: 'Transcription and alignment of the TIMIT database', *DARPA TIMIT CD-ROM Documentation* (1988).
- 15 Wells J: 'Computer-coded phonemic notation of individual languages of the European Community', *J IPA*, **19**, pp 32—54 (1989).

- 16 Hieronymus J L: 'ACSII phonetic symbols for the world's languages: Worldbet', (unpublished).
- 17 Johansson S et al: 'The tagged LOB corpus: User's manual', Norwegian Computing Centre for the Humanities, Bergen (1986).
- 18 Spiegel M and Macchi M: 'Development of the orator synthesiser for network applications', Proc AVIOS (1990).
- 19 Schmidt M and Jack M A: 'A multi-lingual pronunciation dictionary of proper names and place names: ONOMASTICA', Proc ELSNET Language Engineering Conv, pp 125—128 (1994).
- 20 Nirenburg S et al: 'Machine translation — a knowledge-based approach', Morgan Kaufmann, San Mateo, California (1992).
- 21 Charniak E: 'Statistical language learning', MIT Press (1993).
- 22 Viterbi A J: 'Error bounds for convolutional codes and an asymptotically optimal decoding algorithm', IEEE Trans Information Theory, 13, pp 260—269 (1967).
- 23 Bod R: 'Using an annotated corpus as a stochastic grammar', Proc EACL '93, Utrecht (1993).
- 24 Magerman D M and Marcus M: 'Pearl: a probabilistic chart parser', Proc EACL '91, Berlin (1991).
- 25 Kay M: 'Algorithm schemata and data structures in syntactic processing', in Grosz, Sparck-Jones and Lynn-Webber (Eds): 'Readings in natural language processing', Morgan Kaufmann, Los Altos, pp 35—70 (1986).
- 26 Goldsmith J A: 'Autosegmental and metrical phonology', Blackwell, Oxford (1990).
- 27 Durand J: 'Generative and non-linear phonology', Longman, London (1990).

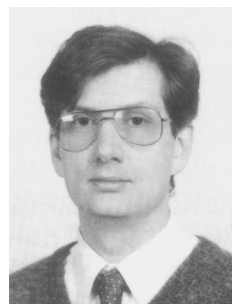


Mike Edgington graduated from the University of York with an MEng degree in electronic systems engineering in 1990. He stayed at York to research towards a DPhil in speech analysis and synthesis models, which he is currently completing.

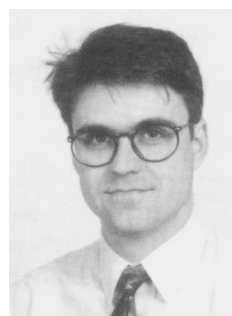
He joined the speech synthesis and analysis group at BT Laboratories in 1993, where he has been involved in many aspects of text-to-speech synthesis research and development, in particular, morphology, pronunciation and multilingual issues. His current research interests include speech modification algorithms and the use of semantic information in TTS. He is a Member of the IEEE and an Associate Member of the IEE.



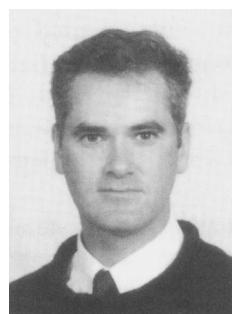
Andrew Lowry received the BSc and PhD degrees in electrical and electronic engineering from Queen's University, Belfast in 1984 and 1988 respectively. He joined BT Laboratories in 1988 to work on format analysis and synthesis of speech, and has been primarily involved in developing signal processing algorithms for various aspects of text-to-speech synthesis. Current research interests include intonation modelling for TTS, and speech analysis using nonlinear dynamics and higher order statistics. He is a Chartered Engineer and a Member of the IEE.



Andrew Breen holds a BSc in Physics with Computing Physics from University College Swansea, an MSc (Eng) in Electrical Engineering from Liverpool University and a PhD in Speech Science from University College London. After completing his first degree he was employed in the department of Electronics and Electrical Engineering at Liverpool University where he worked for three years in the area of speech coding and digital filter design. After leaving Liverpool he was employed as an experimental officer in the department of Phonetics and Linguistics at UCL where he worked on speech synthesis and analysis. On joining BT in 1989 he worked for two years on continuous speech recognition before moving into the area of text-to-speech synthesis. He is project leader of the language component of the Laureate TTS system and project manager of the Synthetic Persona project. His current research interests include speech synthesis, speech prosody and the representation of linguistic and para-linguistic information in a synthetic persona. He is a member of the IEE.



Peter Jackson graduated from UMIST with an MSc in electrical engineering and electronics in 19983. He then joined BT Laboratories, where he initially worked on the electronic design of consumer products. He has also worked on development of the BT range of modems and in the area of speech applications. His current work includes the development of prosody modification and joining algorithms for TTS.



Steve Minnis joined BT's software engineering division in 1987 after graduating with an honours degree in computer science at the Queen's University of Belfast. He was initially involved in software design methodologies and software quality metrics. Later, a spell as a contractor working on machine translation in Japan, at Mitsubishi electric research laboratories, led to an interest in natural language processing. He rejoined BT in 1992, in the newly formed language group, looking at parsing for text-to-speech, and parsing techniques in general.

His current work is research new methods for generation of appropriate prosody for text to speech. Other research interests include lip-motion synthesis for automatic talking heads, and sign language translation. He is currently studying for a part-time PhD in prosodic parsing at York University.